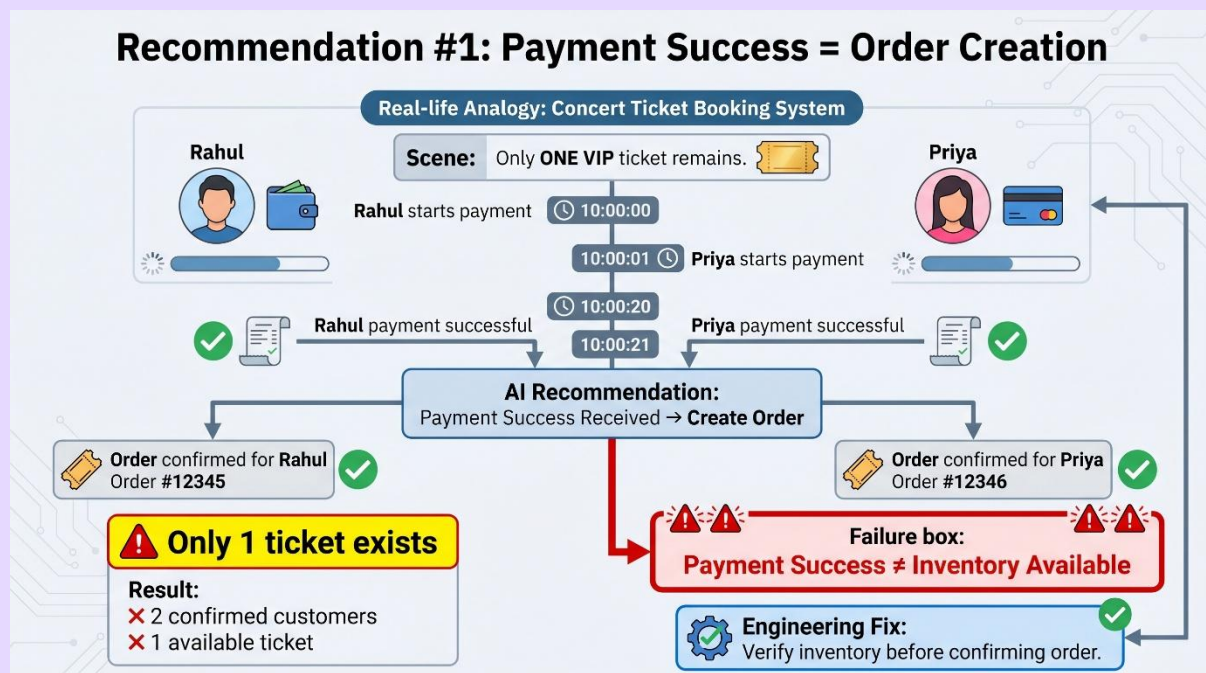
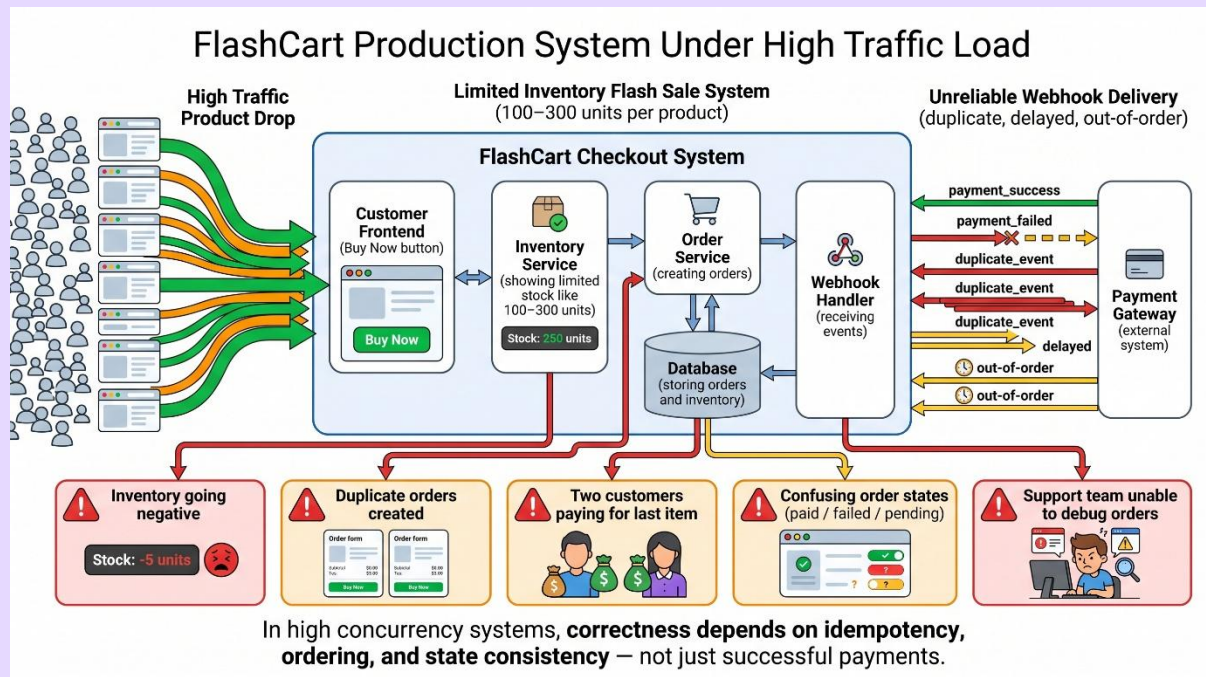
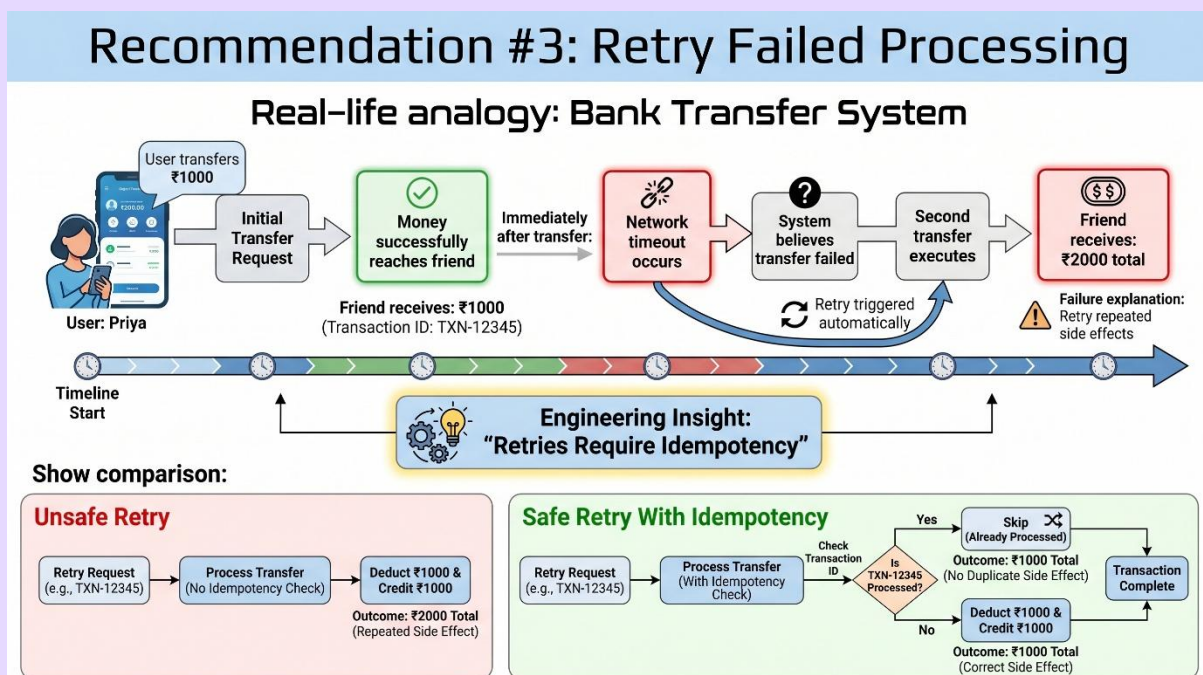
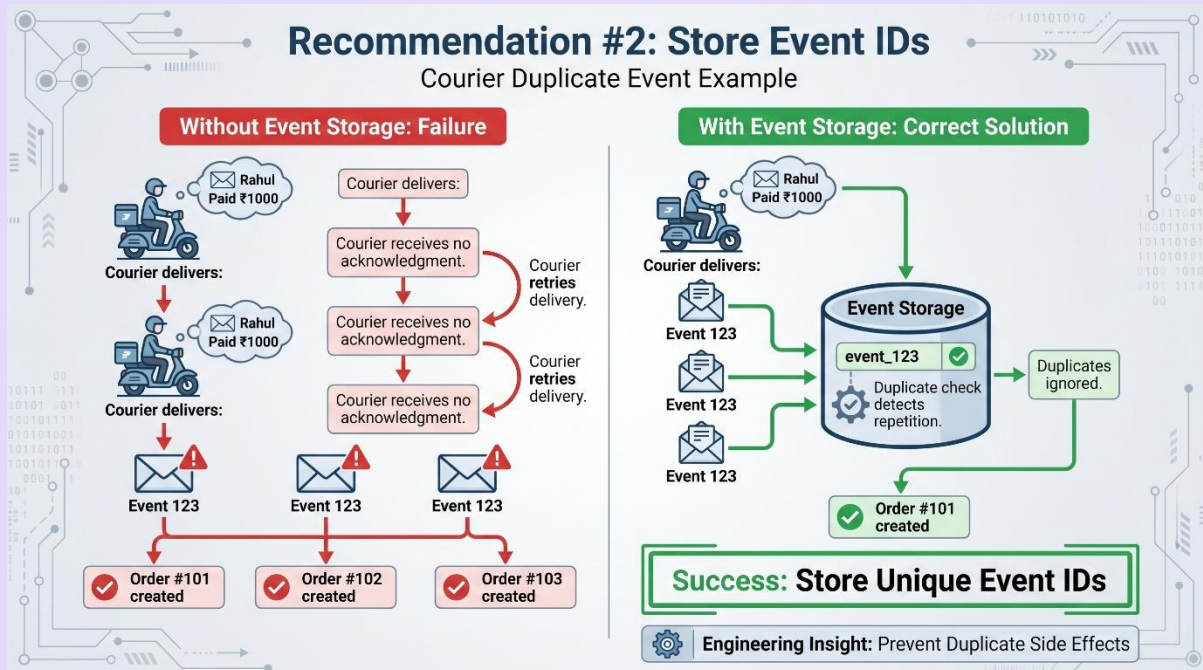


Student: **Gourik Narendra Patil**

Assigned Pack: **D5-B – Payment Webhook AI Plan**





Recommendation #4: Use A Queue

Real-life analogy: Restaurant Order Processing

Scene:

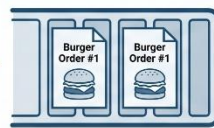
Customer orders one burger



Customer orders one burger

Waiter accidentally submits order twice

Order Queue



Order Queue contains:
Burger Order #1,
Burger Order #1



Chef cooks two burgers

Show highlighted warning:
Queue processed duplicates correctly.
But duplicate data still existed

Engineering Insight:
Queue Changes Timing, Not Correctness

Include side-by-side comparison:

Queue Alone



Result: Duplicates Processed

Queue + Idempotency



Result: Duplicates Handled (Only One Burger Cooked)

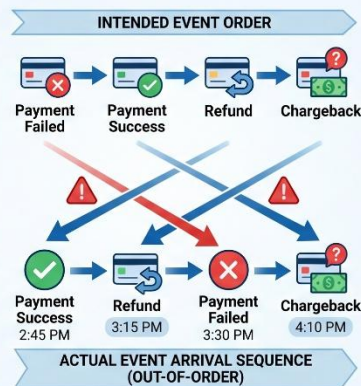
Recommendation #5: Ignore Out-of-Order Events

Real-life Analogy: Postal Mail Delivery



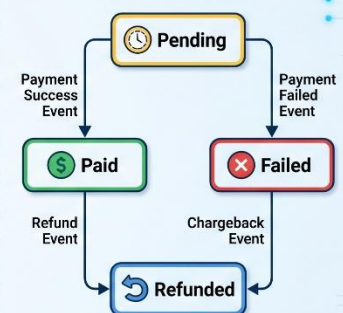
The sequence of arrival (B, then A) contradicts the sequence of sending (A, then B).

Map to Payment Events

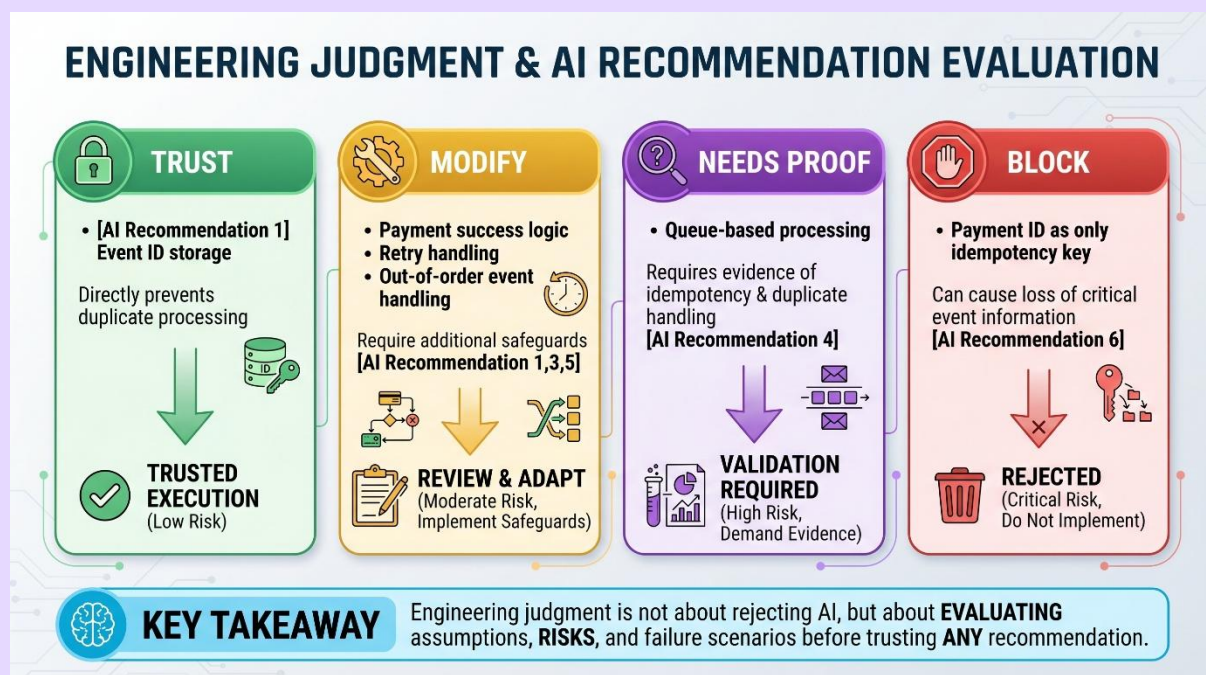
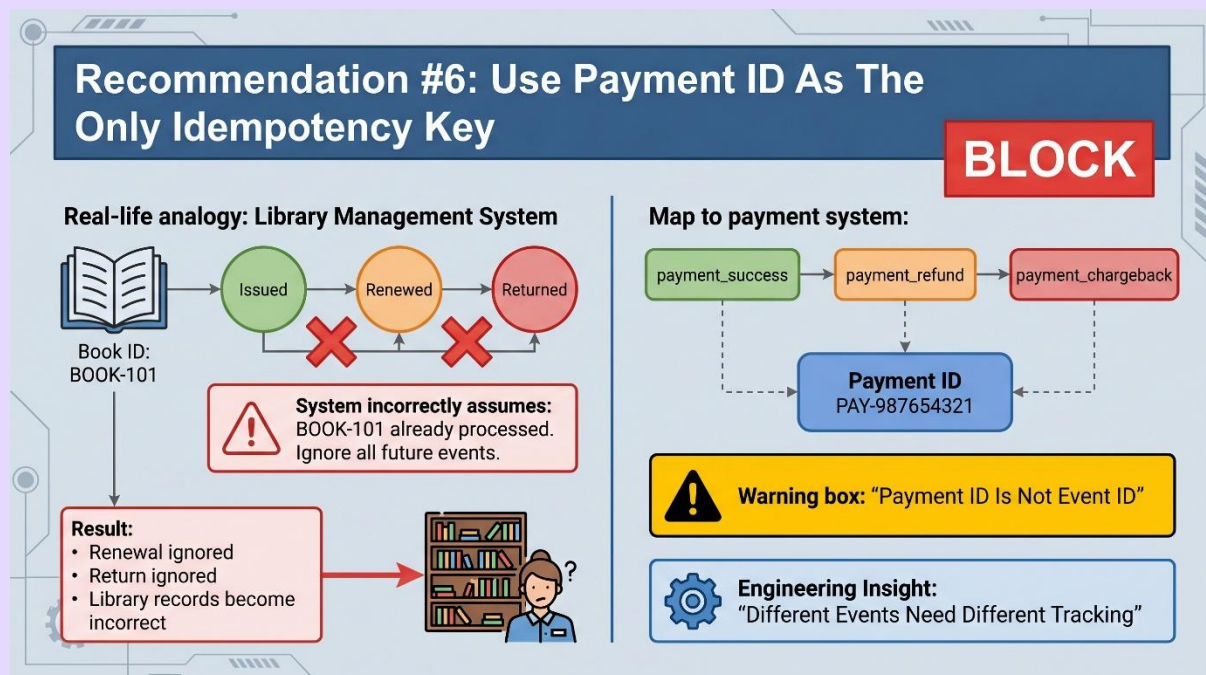


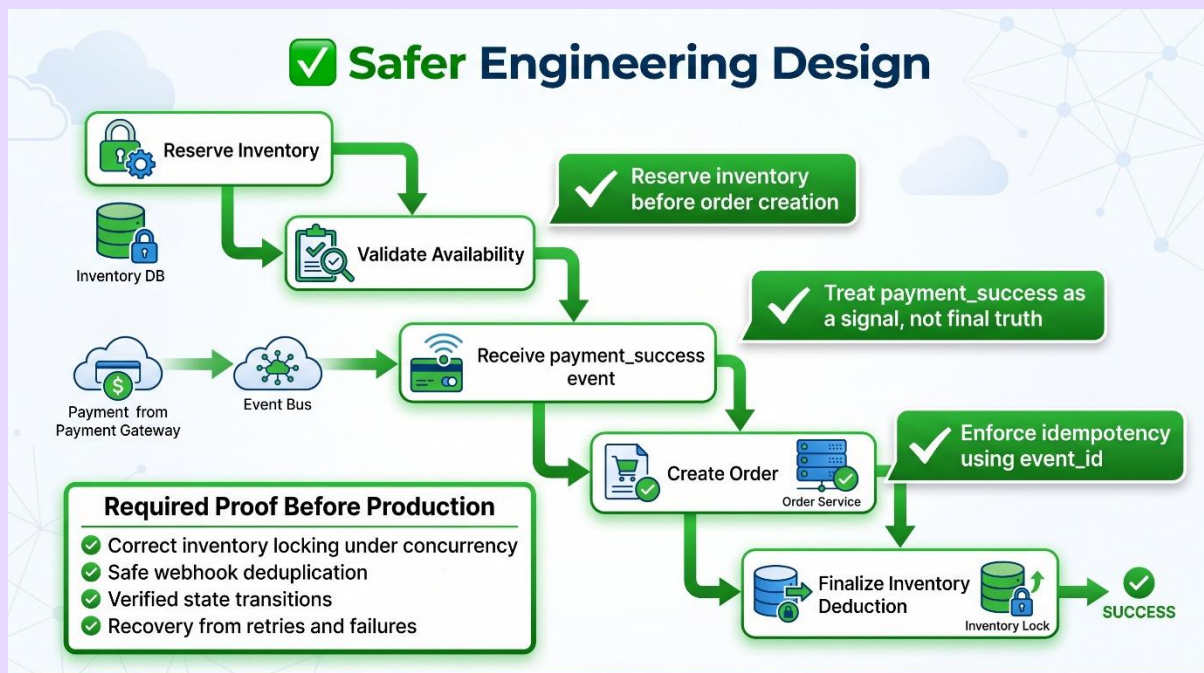
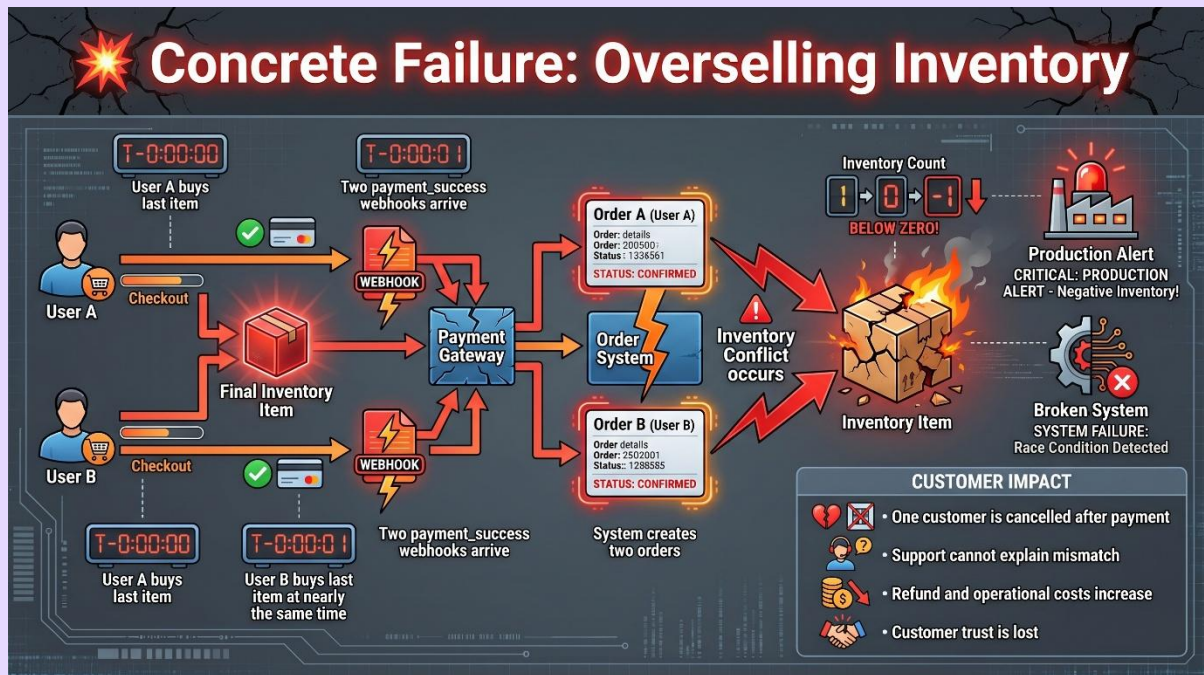
WARNING:
Arrival Order ≠ Event Truth

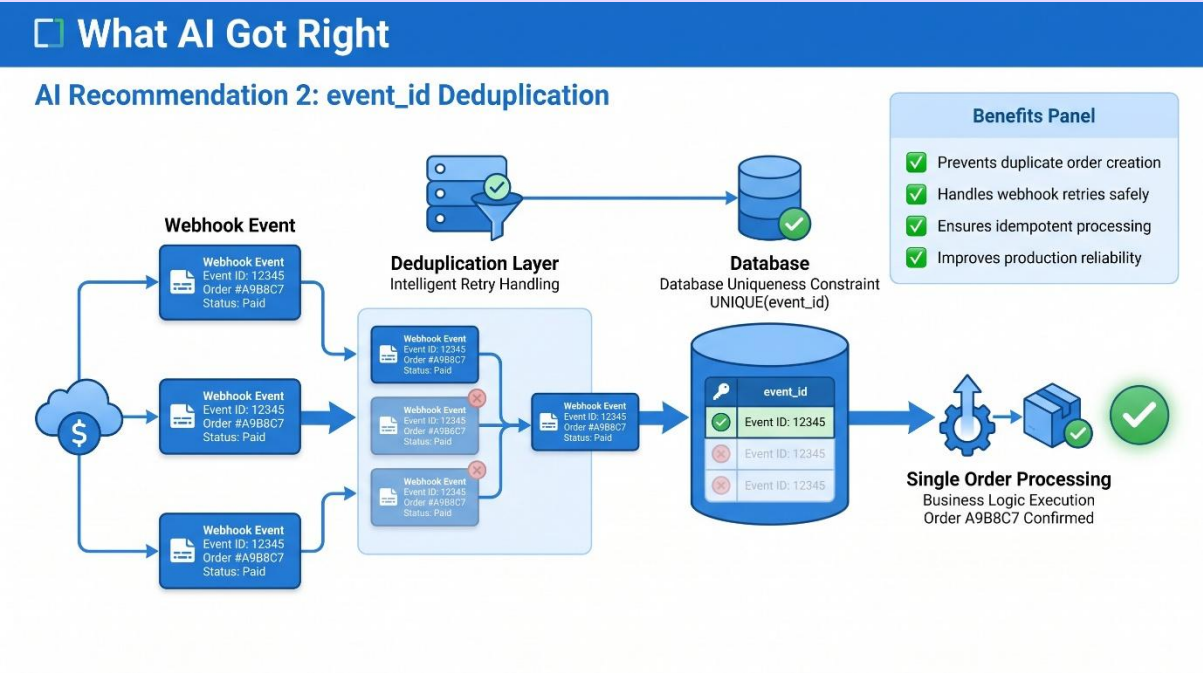
Engineering Insight: Use State Machines, Not Assumptions



State machines ensure events are processed based on current state, not arrival time. Invalid events (e.g., 'Payment Failed' arriving when state is already 'Paid') are correctly ignored or logged, preserving system integrity.







Final Engineering Recommendation

Use a Modified AI-Generated Plan

LEFT COLUMN — AI Got Right

Event tracking

Retry handling

Webhook monitoring

Deduplication concepts

RIGHT COLUMN — Engineering Corrections Required

Inventory availability verification

State-transition safety

Concurrency control

Idempotency validation

AI generated useful ideas, but engineering judgment is required before production deployment.

AI sounded correct, but missed state-transition safety and verification requirements. Inventory protection, duplicate prevention, and reliable order processing must be validated before production.

Table of Contents

1. AI Trust Decision Brief	8
1. Problem Summary	8
2. AI Recommendation Decision Table	9
3. Most Dangerous Recommendation	10
4. Useful But Incomplete Recommendation	10
5. One Recommendation Trusted For V1	11
6. Safer V1 Plan	11
7. State / Side Effect Protection	12
8. What Not To Build	13
9. Evidence Needed Before Trust	13
10. Final Founder Recommendation	14
11. Final One-Liner	14
2. AI Recommendation Classification Matrix	15
Useful AI Ideas	15
Unsafe AI Claims	15
Proof Required	15
3. Safer Engineering Action Plan	16
Recommended V1 Action	16
Modify / Block List	16
Production Failure Prevented	16
Verification Required	17
Founder-Ready Recommendation	17

1. AI Trust Decision Brief

Student: Gourik Narendra Patil

Assigned Pack: D5-B – Payment Webhook AI Plan

1. Problem Summary

FlashCart is a high-traffic e-commerce system with limited inventory drops, where thousands of users compete for 100–300 units. This has led to production issues such as duplicate webhook processing, negative inventory, multiple customers purchasing the last available item, and inconsistent order states.

The AI-generated plan attempts to improve reliability using webhook event tracking, retries, queue-based processing, and idempotency mechanisms. My responsibility is to evaluate which recommendations are safe for a one-week v1 release, which require modification, and which are unsafe without further evidence.

The key engineering concerns are:

- Inventory correctness under concurrency
- Webhook idempotency
- Event ordering reliability
- Safe state transitions between payment and order creation

2. AI Recommendation Decision Table

#	AI Recommendation	Decision	Why	Concrete Failure If Wrong	Safer Action / Evidence Needed
1	Trust payment_success webhook as order truth	Modify	Payment success does not guarantee inventory availability	Two users can be confirmed for the last item	Reserve inventory before order creation
2	Store gateway event IDs	Trust	Prevents duplicate webhook processing	Duplicate orders and duplicate inventory deduction	Store event_id with DB unique constraint
3	Retry failed webhook processing	Modify	Retries are useful but unsafe without idempotency	Same operation executed multiple times	Retry only after idempotent processing
4	Process webhooks in a queue	Needs Proof	Queue improves throughput but not correctness	Duplicate events still processed	Prove consumer idempotency + failure handling
5	Ignore out-of-order events	Modify	Some late events are valid state transitions	Refunds or chargebacks may be ignored	Use explicit state machine transitions
6	Use payment_id as only idempotency key	Block	Different event types share same payment_id	Important events like refund ignored	Use event_id for processing idempotency

3. Most Dangerous Recommendation

Recommendation:

Use payment_id as the only idempotency key.

Why it sounded correct

Payment IDs are assumed to be unique per transaction, so it appears safe for deduplication.

Failure Timeline

First: Payment success webhook is processed with payment_id = P123.

Then: System stores P123 as processed.

Then: A refund or chargeback event arrives with the same payment_id.

Then: System ignores it, assuming it is duplicate.

Production breaks because:

Valid financial lifecycle events are skipped.

Customer / support impact:

- Refunds not reflected in system
- Financial records become inconsistent
- Support cannot explain payment state mismatch

What I Would Do Instead

- Use **event_id** for webhook idempotency
- Use **payment_id** for transaction grouping only
- Enforce **state validation before applying transitions**

4. Useful But Incomplete Recommendation

Recommendation: Retry failed webhook processing three times

What was useful

Retries help recover from temporary network failures and database downtime.

What was missing

Retries may repeat side effects if idempotency is not enforced.

Modified Version

Retries should only be applied after:

- event-level idempotency is implemented
- state transitions are validated
- database transactions are safe

5. One Recommendation Trusted For V1

Recommendation: Store gateway event IDs

Why it is safe enough

Duplicate webhook delivery is a real and frequent issue in payment systems. Event ID tracking is a proven and lightweight safeguard.

Boundary condition

- event_id must be stored in a relational database
- must have a UNIQUE constraint
- must be checked before processing webhook

6. Safer V1 Plan

Inventory Ownership

Relational database is the single source of truth.

Checkout / Reservation State

- CREATED
- RESERVED
- PAID
- FAILED
- EXPIRED

Payment State

- PENDING
- PAID
- FAILED
- REFUNDED

Order Confirmation Rule

Order is confirmed only if:

- Payment is successful
- Inventory is reserved
- Inventory is available

Duplicate / Retry Handling

- Store unique event_id
- Ensure idempotent webhook processing
- Use transactional updates for state changes

Expiration / Release Behavior

- Expired reservations automatically release inventory

Customer / Support Visibility

Every checkout must clearly expose:

- RESERVED
- PAID
- FAILED
- EXPIRED
- REFUNDED

7. State / Side Effect Protection**What must NOT happen twice**

- Order creation
- Inventory deduction
- Payment processing

Protected Transition

RESERVED → PAID → ORDER_CONFIRMED

Protection Mechanisms

- UNIQUE(event_id) constraint
- Database transactions
- State validation before transitions

What breaks under concurrency

- Duplicate orders
- Negative inventory
- Double payment processing

8. What Not To Build

Not In V1	Why Not Now	Risk
Full microservices architecture	Too complex for 1 week	Delays delivery
Event sourcing system	Over-engineering	Hard to maintain
Dedicated inventory service	Small team constraint	Operational overhead
Cache as source of truth	Unsafe for inventory	Data inconsistency

9. Evidence Needed Before Trust

Claim	Evidence Needed
Queue improves reliability	Duplicate-event load testing
Out-of-order events can be ignored	Gateway documentation
payment_id prevents duplicates	Tests with refund + chargeback cases

10. Final Founder Recommendation

Decision: Use modified AI plan

Founder Update

The AI plan contains useful ideas such as event tracking and retry handling. However, it is unsafe as written because it assumes inventory availability, correct event ordering, and idempotency without verification.

I recommend a modified v1 system that enforces:

- event-level idempotency
- inventory reservation before order confirmation
- explicit state transitions
- safe retry handling
- clear customer-visible order states

11. Final One-Liner

AI sounded correct, but it missed **state-transition safety and event-level idempotency**, so I changed the design to ensure inventory protection, prevent duplicate side effects, and guarantee reliable order processing.

2. AI Recommendation Classification Matrix

Student: Gourik Narendra Patil

Variant ID: D5-B

AI Recommendation	Classification	Evidence / Risk	Safer Boundary
Payment success as order truth	Modify	Ignores inventory constraints	Validate reservation first
Store gateway event IDs	Trust	Prevents duplicates	DB unique constraint
Retry webhook processing	Modify	Can repeat side effects	Idempotent retry only
Queue processing	Needs Proof	Does not ensure correctness	Prove idempotent consumer
Ignore out-of-order events	Modify	Valid events may be lost	State machine logic
Payment ID as only key	Block	Skips lifecycle events	Use event_id

Useful AI Ideas

- Event-based deduplication
- Retry mechanism concept
- Async processing consideration

Unsafe AI Claims

- Payment success equals order finality
- payment_id alone ensures idempotency
- Event order can be ignored safely

Proof Required

- Queue failure simulation tests
- Duplicate webhook injection tests
- Event ordering behavior documentation

3. Safer Engineering Action Plan

Student: Gourik Narendra Patil

Variant ID: D5-B

Recommended V1 Action

Implement:

- event-level idempotency
- inventory reservation before payment confirmation
- strict state transitions
- safe retry handling
- relational DB as single source of truth

Modify / Block List

AI Suggestion	Action	Why
Payment success creates order	Modify	Inventory must be verified first
Retry processing	Modify	Needs idempotency
Queue processing	Needs Proof	Does not ensure correctness
Ignore out-of-order events	Modify	Can lose valid transitions
Payment ID as key	Block	Breaks lifecycle tracking

Production Failure Prevented

- Two users buy last item simultaneously
- Both payments succeed
- Two orders are created
- Inventory goes negative
- System shows inconsistent state

Verification Required

Check	Evidence Needed
Queue safety	Load + duplicate tests
Retry safety	Idempotency validation
Event ordering	Gateway documentation

Founder-Ready Recommendation

The AI plan contains useful operational improvements, but it is not safe for direct production use. Event-based deduplication is valid and should be implemented. However, payment success must not directly create orders without inventory verification. Queue processing requires proof of correctness under duplicate and failure conditions. For v1, the system should prioritize inventory protection, event-level idempotency, transactional safety, and clear state transitions.